USB chat

This file stays at the **repository root** so GitHub can show it. The other project notes live in **‘documentation/‘** (documentation/):

- **Changelog** (documentation/CHANGELOG.md)
- **Requirements** (documentation/REQUIREMENTS.md)
- **Bench results** (documentation/TestResults.md)
- **Quality test results** (documentation/Quality Test Results.md)

The license is **‘LICENSE‘** (LICENSE) in this folder, next to `.git` and `.gitignore`. Those stay at the repository root. They are not copied to the USB.

The folder layout below is for the **USB**, not a second copy of this repository. On the stick, do not leave Markdown at the root. Systems that open the stick often have no Markdown viewer, so the root exposes the PDFs only (`README.pdf`, `CHANGELOG.pdf`, `REQUIREMENTS.pdf`, `TestResults.pdf`). The Markdown, including a copy of this file, goes in `documentation/`. Setup rebuilds the guide PDFs from that Markdown. Setup and chat do not open the PDFs.

It is **not** the id10t factory (no mail, no Serp/Scraper keys, no second GGUF). Coding uses the model you picked, one at a time.

Features below are what **1.2.20** can do.

Features

- One GGUF at a time. The picker lists only files that fit **free RAM >= file size x 1.2 + 2 GB**, largest first, labeled like `Qwen2.5 Coder 14B (8.4 GB)`. If the file’s chat template rejects a system role, that text is folded into the first user message.
- Chat window: labeled History (empty-state hint) and Message (placeholder + Send on the right). While the model is working (including a long Web Search), the box is locked and Send becomes a spinner; Stop stays on.
- Quote a turn with the control on each **You** / **Model** line (hover: *Reply to this message*). **Clear quote** and **Replying to:** appear only while a quote is pending.
- Read-only attach of one file, or **Attach folder...** for a folder of `pdf`, `docx`, `xlsx`, `txt`, `md`, and `csv`. Other files in that folder are skipped. The folder is read before the model loads, and a long folder is walked in parts so it fits the context window. History shows a document icon and the full path (older logs: icon and the stored name).
- Session logs in `logs/` (JSONL with prompt, optional `shown` / `attached`, reply, token counts, and Web Search fields when used). Scroll the transcript to the top to load an older log as earlier llama.cpp turns. When none remain, History says nothing is stored before this.
- Logs older than 30 days are deleted by the `YYYY-MM-DD_` prefix in the filename (session `.log` and `web_*.jsonl`).

- `notes.md` is included (first 4000 characters). The model cannot write it. **Notes...** edits the file; each Model reply has **Save to notes** (you confirm).
- History paints headings, lists, and code as widgets (`formatter.py`). **View HTML** on a reply opens a sanitized temp file in the OS browser. Footer **Save transcript PDF** / **Print** still export the whole chat. A per-reply **Save PDF** appears only when that turn wrote a PDF into `outputs/`.
- **Coding** checkbox above Message. Optional **GitHub URL** and **Folder...** show only while Coding is on (public `https://github.com/owner/repo`, or a local folder copied into `outputs/`; not both). See Coding below.
- Optional **Web Search** - see below. Still off unless you turn it on.
- Host `setup.bat` / `setup.sh`: require **Python 3.11+** (see **‘documentation/REQUIREMENTS.md‘** (documentation/REQUIREMENTS.md)). They create `logs/`, `coding-logs/`, and `outputs/` on the stick (and stop if the stick is not writable). Compatible interpreters only seed a **private host venv** (`%LOCALAPPDATA%/usb-chat/venv` or `~/.local/share/usb-chat/venv`); installs `llama-cpp-python` (CPU wheel), **‘pypdf‘**, **‘requests‘**, and **‘beautifulsoup4‘**. A failed install names that library. Missing **git** is a notice, not a failure. System Python is never replaced. If the PC has no 3.11+ (or only 2.7 / 3.9), a dialog names the version we need and the version we found. `chat.bat` / `chat.sh` boot that venv.
- Windows first; same scripts on Ubuntu.
- No mail and no SerpAPI / ScraperAPI keys.

Coding

Leave **Coding** off for chat and for simple snippets. The picked model still writes code; it is marked **not tested**. If you ask it to run tests or use a sandbox with Coding off, it says it cannot test.

Turn **Coding** on to run a sequential committee on that same GGUF (one role at a time, a few rounds). Host ownership rules stay on the board: coder writes implementation files plus `main.py`; the host compile+imports the libraries and then runs `main.py` until that process exits 0 (syntax/import/run errors, not quality). After a seed folder is copied, the host scans non-script documents (`data/*.csv` and similar), builds a **data card** once, and passes it to the planner at phase 0; the manager may write a `test_plan` from those cell values, then it is locked. A write that drops locked inventory names is rejected (`main.py` may be rewritten). Tester then writes/runs only `test_*.py` / `*_test.py`, must clone the `main.py` call pattern with other literals that still appear on the data card, and must call the host **live inventory** names (coder-owned APIs, merged each round; writes that invent other names, fail to compile, or hardcode a `SCORES 1,1,1` list are rejected). On an impl crash, the coder's next write must be the file named in the traceback. The tester loop writes and runs tests until the harness prints n computed `SCORES` values or the crash is in impl; a `test_broken` script is not re-executed until it is rewritten. Later rounds re-execute only after a real score line. Critique sees a results packet (not log dumps). Evaluator plus a host file/function inventory drop "missing" hallucinations before `next_fix`. Manager writes the contract once from the prompt, then only appends fix versions from newest critiques and a done-list. Qualify reads the last `SCORES metric=v1,v2,...` line (average of `n` trials). Work lands in a folder under `outputs/` on the stick. Each committee run writes a folder under `coding-logs/` (`generic.jsonl`, `planner.jsonl`, `critique.jsonl`, `test.jsonl`; append each loop; mirrored in the work folder as `_logs/`). That folder is not History and is not auto-deleted; remove it yourself when you want it gone. A GitHub URL is cloned only when Coding is on, the URL is a public GitHub repo, and `git` is on the PC. **Folder...** copies a local project into the work folder instead (not in-place; skip `.env` / venvs / `.git`). `TASK.md`, `PROMPT.md`, or `README.md` in that folder is appended to the Message text. No tokens. Stop cancels between roles.

High-stakes wording (legal, official, medical, and similar) without Coding uses a short review pass that may write a document or PDF. It does not execute code.

Web Search

Shown above Message only when the host can reach the internet. The toggle stays on until you turn it off. Hover the asterisk: it is slow and uses extra model passes; use it only when you need live web.

With the toggle on, **Send** runs this local path (`search.py`, no paid search API):

1. The model writes a Google query from the latest request.
2. `search_google` fetches `google.com/search` HTML and strips the cookie/consent chrome.
3. The model names CSS classes/ids to **remove** or **keep** (a keep pair is class and id on the same tag).
4. The model picks at most **3** http(s) URLs (Google wrapper links are unwrapped; google.com chrome is skipped). If it names none, the scrape falls back to links found in the HTML.

5. Each page is fetched, ads/search boxes stripped, then the same remove-or-keep pass.
6. The model answers from those excerpts. **Only that last reply is shown in History.**

If Google HTML or page text will not fit the local context window (default **4096** tokens, minus a **512**-token reply), it is split and processed in parts. Status can show `cleaning results (2/5)` or `summarizing page 1, part 2/3`. Chunk notes stay out of History. If a fetch is blocked or empty, the turn answers without the web and says so in the status line.

Stop cancels between steps. Query, raw HTML sizes, class/id specs, URLs, and summaries go to `logs/web_YYYY-MM-DD_HHMMSS.jsonl`, not the transcript. Session turns may store `web_search`, `web_query`, `web_urls`, and `web_log`.

Needs `requests` and `beautifulsoup4` in the host venv (setup installs them). Google HTML can still consent-wall or block; that is expected for a raw scrape.

GPU offload is off unless you set `USB_CHAT_N_GPU_LAYERS` (or a numeric `N_GPU_LAYERS`) on the host.

Layout on the USB

Copies onto the USB must follow this tree. The stick root is home. There is no `chat/` folder. Markdown is not part of that home directory; only the PDFs are.

```
<USB>/
  chat.bat
  setup.bat
  chat.sh
  setup.sh
  README.pdf
  CHANGELOG.pdf
  REQUIREMENTS.pdf
  TestResults.pdf
  notes.md             (created on the stick; do not put in git)
  *.gguf
  scripts/
    chat.py
    usbchat.py
    setup.py
    committee.py
    formatter.py
    search.py
    bench.py
    download_models_local.py
    bin/               (optional llama-cli)
    seeds/             (bench tasks)
    frozen/            (frozen bench snapshot)
    tests/
  documentation/
    README.md
    CHANGELOG.md
    REQUIREMENTS.md
    TestResults.md
    Quality Test Results.md
```

```
  logs/
    YYYY-MM-DD_HHMMSS.log
    web_YYYY-MM-DD_HHMMSS.jsonl   (Web Search side log; not History)
  coding-logs/         (committee traces; not History; delete by hand)
    YYYY-MM-DD_HHMMSS_<slug>/
      generic.jsonl    (every role complete/tool)
      planner.jsonl    (phase0 + each manager plan; append)
      critique.jsonl   (each critique round; append)
      test.jsonl       (writes, executes, qualify, ballot; append)
  outputs/             (coding/review work folders and bench run folders; not git)
```

`models.json` on the stick is unused by this chat. `logs/`, `coding-logs/`, `outputs/`, and `notes.md` stay on the USB only. Run products go in `outputs/` only. On the stick, project Markdown stays in `documentation/` and the four PDFs stay at the root.

This repository is not that tree. `README.md`, `LICENSE`, `.git`, and `.gitignore` stay at this repo root. `documentation/` holds the other Markdown. `scripts/` holds the Python that gets copied into `scripts/` on the stick. Do not copy `LICENSE`, `.git`, or `.gitignore` onto the stick.

Copy onto the stick

Copy `scripts/` and `documentation/` onto the stick, and copy the four PDFs plus `chat.bat`, `setup.bat`, `chat.sh`, and `setup.sh` to the stick root, beside the GGUFs. Put this `README.md` in `documentation/` on the stick, not at the stick root. Leave `LICENSE`, `.git`, and `.gitignore` in the repo only. Do not copy `.venv`, `logs/`, `coding-logs/`, `outputs/`, `notes.md`, or `*.gguf` into git. Do not put the scripts back into a `chat/` folder.

On a **new** stick: put the GGUFs at the USB root, copy the folders as above, then run setup from the stick root.

What the host needs (runtime, on the USB)

See **‘documentation/REQUIREMENTS.md‘** (documentation/REQUIREMENTS.md). On the stick, the visual copy is `REQUIREMENTS.pdf` at the root. Short version: Python **3.11+**, a writable stick, `llama-cpp-python` or `llama-cli`, plus `pypdf` / `requests` / `beautifulsoup4` from setup. **git** is optional.

From the stick root:

```
./setup.bat
./chat.bat
```

```
chmod +x setup.sh chat.sh
./setup.sh
./chat.sh
```

Optional: `--models D:/` if the GGUFs are not at the stick root and not on `E:/`...`P:/`.